

Developing a Domain-Specific LLM through Self-Instruct Fine-Tuning

Application to Reliability Engineering

Alex Dalban Elora Drouilhet

CentraleSupélec

Supervisor: Dr. Zhiguo Zeng

December 12, 2025

Outline

- 1 Introduction & Motivation
- 2 Project Goals & Methodology
- 3 Literature Review
- 4 Data Pipeline
- 5 Results & Analysis
- 6 Discussion & Future Work
- 7 Conclusion

Reliability Engineering Challenges

- Complex systems: nuclear plants, aircraft, electrical grids
- Requires sophisticated mathematical modeling
- Significant manual programming effort
- Specialized domain knowledge needed

General LLMs Fall Short

- Built for general programming/tasks
- Poor performance on reliability-specific tasks
- Lack of specialized training data
- Cannot reason through multi-step domain problems

Key Issue

State-of-the-art LLMs consistently provide **incorrect or nonsensical code** for reliability engineering tasks due to lack of domain exposure during pre-training.

Previous Work & Baseline

- Previous students created **58 hand-written reliability coding tasks** with unit tests
- Evaluated state-of-the-art LLMs on this benchmark
- Results were consistently poor across all models

We ran our own baselines on reliability engineering textbooks problems:

Baseline Performance1 (Unfinetuned Qwen3-14B)

REDO THIS **5/58 correct = 8.6% accuracy**

Baseline Performance2 (Unfinetuned Qwen3-14B)

REDO THIS **5/58 correct = 8.6% accuracy**

Common Failure Modes:

- Lack of domain specific formulas
- Hallucinated reasoning steps/formulas

Primary Goal

Develop a highly effective, domain-specific LLM (reasonably sized) for reliability engineering through fine-tuning with synthetic data, I DONT EVEN KNOW FOR HIS LOL

Three Key Components: maybe rework this:

① Synthetic Data Generation

Use self-instruct strategy to automatically generate large corpus of Q&A pairs

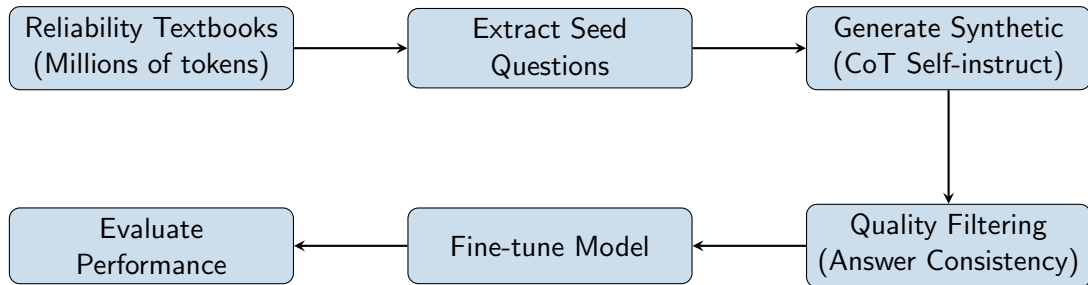
② Chain-of-Thought Integration

Generate step-by-step reasoning for complex reliability problems

③ Model Fine-Tuning & Evaluation

Fine-tune pre-trained LLM and rigorously evaluate performance

Methodology Overview



Self-Instruct & Synthetic Data Generation

Self-Instruct (Wang et al., 2023)

Pioneering framework for bootstrapping instruction-following capabilities from LLMs themselves. Uses seed demonstrations to generate diverse synthetic instruction-output pairs, achieving 33% improvement over vanilla GPT-3 on Super-NaturalInstructions.

CoT-Self-Instruct (Yu et al., 2024)

Extends self-instruct by incorporating Chain-of-Thought reasoning during generation. Uses Answer-Consistency filtering for verifiable tasks. Outperforms original 'seed' datasets (s1k, OpenMathReasoning) on MATH500, AMC23, AIME24, and GPQA-Diamond benchmarks.

Relevance to Our Work: We adopt the self-instruct paradigm with consistency-based quality filtering to generate synthetic reliability engineering questions from textbook seeds.

Chain-of-Thought Reasoning

Auto-CoT (Zhang et al., 2022)

Eliminates manual effort in creating CoT demonstrations by using diversity-based question clustering and automatic reasoning chain generation. Matches or exceeds manual CoT performance across 10 reasoning benchmarks with GPT-3.

Key Innovation:

- Cluster questions by similarity
- Sample diverse representatives
- Generate reasoning chains with "Let's think step by step"
- Mitigate errors through diversity

Self-Refine IT (Ranaldi & Freitas, 2024)

Two-stage alignment: (1) Transfer reasoning via instruction-tuning on LLM demonstrations, (2) Self-refinement through Direct Preference Optimization (DPO). Enables smaller models to learn step-wise reasoning without separate reward models.

Results:

- Outperforms standard instruction-tuning
- Strong in/out-domain generalization
- Preference optimization crucial for reasoning quality

Domain-Specific Application: Financial Reasoning

Fin-R1 (Liu et al., 2025)

Domain-specific reasoning LLM for finance using two-stage training: (1) SFT on 60K CoT financial examples distilled from DeepSeek-R1, (2) RL with GRPO (Group Relative Policy Optimization) using format and accuracy rewards. Achieves 7B parameter model with SOTA performance on FinQA and ConvFinQA, rivaling much larger models.

Methodology Parallels:

- Domain-specific dataset curation
- Quality filtering (LLM-as-judge)
- Parameter-efficient training
- Numerical reasoning focus

Key Differences:

- Finance: 60K examples vs our 81
- Uses RL phase (we use only SFT)
- Distills from stronger teacher model
- Structured output format enforcement

Lesson: Domain-specific LLMs require substantially larger datasets (1000s, not 10s) and benefit from RL-based refinement for complex reasoning tasks.

Step 1: Question/Answer Extraction

- Scan textbooks for Example or Problem or Exercise keywords with close-by Solution keyword
- Extract all surrounding text using regex

Step 2: Quality Validation

- Remove samples that have or refer to figure/table references
- Validate self-containment and whether it meets our question requirements
- Verify clear, unambiguous answers

Step 3: Reformatting Data

- Separate text into 3 components using an LLM:
- Question
- Reasoning
- Answer

Result

288 high-quality seed (q, t, a) samples for baseline evaluation and synthetic generation

Synthetic Data Generation

Self-Instruct Strategy:

- 1 Sample 2 seed questions randomly
- 2 Prompt LLM to create novel inspired question
- 3 Generate step-by-step solution with CoT
- 4 Extract new (q, t, a)

Quality Control - Answer Consistency Checking:

- Re-solve generated question n times (with somewhat high temperature)
- Calculate consistency score: matches / total attempts
- Accept only both the score ≥ 0.5 and the majority prediction is equal to initial answer
- Filters out ambiguous or incorrectly formulated questions

Generation Results

Generated **81 synthetic questions** that passed consistency filtering

Baseline Evaluation Results

Model	Dataset	Correct	Total	Accuracy
GPT-4-Mini	Seed	122	288	42.4%
Qwen3-14B	Seed	100	288	34.7%
GPT-4-Mini	Synthetic	74	81	91.4%
Qwen3-14B	Synthetic	77	81	95.1%

Key Observation

Synthetic questions have **much higher** accuracy which makes sense as a similar capacity model created these questions

Open Questions

Does this matter for finetuning? Does a smaller LM even have the capacity to learn questions generated by larger LM?

Fine-Tuning Results

Fine-tuning Setup:

- Model: Qwen3-14B with LoRA adapters
- Training data: 230 Seed Q&A pairs

Results for held out Q&A pairs

Model	Correct	Total	Accuracy	Change
Unfinetuned Qwen	5	58	8.6%	—
Finetuned Qwen	8	58	13.8%	+5.2%

Result

60% relative improvement in accuracy on seed questions shows that the models are able to learn how to solve these problems and finetuning allows the models to better generalize

Challenges Encountered

① Data Quality Gap

- Synthetic questions too different from real problems
- Models perform well on generated data but poorly on textbook questions
- Need better generation strategies to match complexity

② Limited Training Data

- Only 81 synthetic questions after filtering
- Original goal: thousands of examples
- Consistency filtering too strict? (50% threshold)

③ Domain Complexity

- Reliability engineering requires deep statistical knowledge
- Multi-step reasoning with specialized distributions
- Numerical precision matters (small errors = incorrect)

Data Generation

- Increase synthetic dataset size (500+)
- Use textbook context as seed material
- Improve consistency checking
- Generate chain-of-thought solutions
- Add difficulty stratification

Model Training

- Full fine-tuning (not just LoRA)
- Larger base models (70B+)
- Multi-stage training pipeline
- Reinforcement learning from feedback

Evaluation

- Expand test set beyond 58 questions
- Human expert evaluation
- Code execution testing
- Real-world problem benchmarks

System Integration

- Interactive coding assistant
- Auto-correction with verification
- Uncertainty quantification
- Explainable AI for solutions

Proposed Next Steps

Short-term (Next 2 weeks)

- ① Generate 200+ additional synthetic questions
- ② Retrain with larger, more diverse dataset
- ③ Analyze failure modes on incorrectly solved problems
- ④ Implement better answer verification

Medium-term (1-2 months)

- ① Expand to full fine-tuning approach
- ② Create category-specific datasets (e.g., Weibull, exponential distributions)
- ③ Develop interactive evaluation interface
- ④ Prepare manuscript for publication

Key Findings

- ❶ **Problem Confirmed:** General LLMs perform poorly on reliability engineering tasks (8.6% baseline accuracy)
- ❷ **Fine-tuning Works:** Domain-specific training improves performance by 60% relative improvement (8.6% → 13.8%)
- ❸ **Data Quality Critical:** Synthetic questions much easier than real textbook problems (95% vs 14% accuracy)
- ❹ **Scale Matters:** Current dataset too small (81 questions) for robust domain adaptation

Main Conclusion

Self-instruct fine-tuning is a **viable approach** for domain-specific LLM development, but requires **larger, higher-quality** training datasets that better match real-world problem complexity.

Contributions

Technical Contributions

- Automated Q&A extraction pipeline from textbooks
- Self-instruct synthetic generation with quality filtering
- Baseline evaluation framework for reliability engineering
- Open-source codebase for domain-specific LLM development

Scientific Contributions

- First systematic study of LLMs for reliability engineering
- Quantified performance gap: general vs domain-specific
- Demonstrated feasibility of synthetic data approach
- Identified key challenges for future research

Thank you for your attention!

Contact:

Alex Dalban & Elora Drouilhet
CentraleSupélec - LGI Laboratory

Supervisor:

Dr. Zhiguo Zeng
`zhiguo.zeng@centralesupelec.fr`

Slides and code available on GitHub

References & Resources I

Key Papers:

- ① Wang, Y. et al. (2023). *Self-Instruct: Aligning Language Models with Self-Generated Instructions*. ACL 2023.
- ② Zhang, Z. et al. (2022). *Automatic Chain of Thought Prompting in Large Language Models*. ICLR 2023. arXiv:2210.03493.
- ③ Ding, B. et al. (2024). *Data Augmentation using Large Language Models: Data Perspectives, Learning Paradigms and Challenges*. ACL 2024 Findings. arXiv:2403.02990.
- ④ Ranaldi, L. & Freitas, A. (2024). *Self-Refine Instruction-Tuning for Aligning Reasoning in Language Models*. EMNLP 2024. arXiv:2405.00402.
- ⑤ Yu, P. et al. (2024). *CoT-Self-Instruct: Building High-Quality Synthetic Data for Reasoning and Non-Reasoning Tasks*. arXiv:2507.23751.
- ⑥ Liu, Z. et al. (2025). *Fin-R1: A Large Language Model for Financial Reasoning through Reinforcement Learning*. arXiv:2503.16252.

Models Used:

- Qwen3-14B (Alibaba Cloud)
- GPT-4-Mini (OpenAI)
- Mistral-Small-3.1-24B (Mistral AI)

Tools & Libraries:

- OpenRouter API
- Python (extractors, evaluators, generators)
- LoRA (Low-Rank Adaptation)